

QUANTICS 2026

# Co-evolutionary Asymmetry vs. Modular Design

---

Optimization Trade-offs in Quantum Denoising Autoencoders

**J. L. Cybulski · J. Zwoniariski · A. Strąg · S. Zając**

Deakin University · Military University of Technology · Quantum AI Foundation

# What we analyze: noisy, chaotic time series

The starting point is practical: we have a signal evolving in time, measured imperfectly. We want the clean signal back.

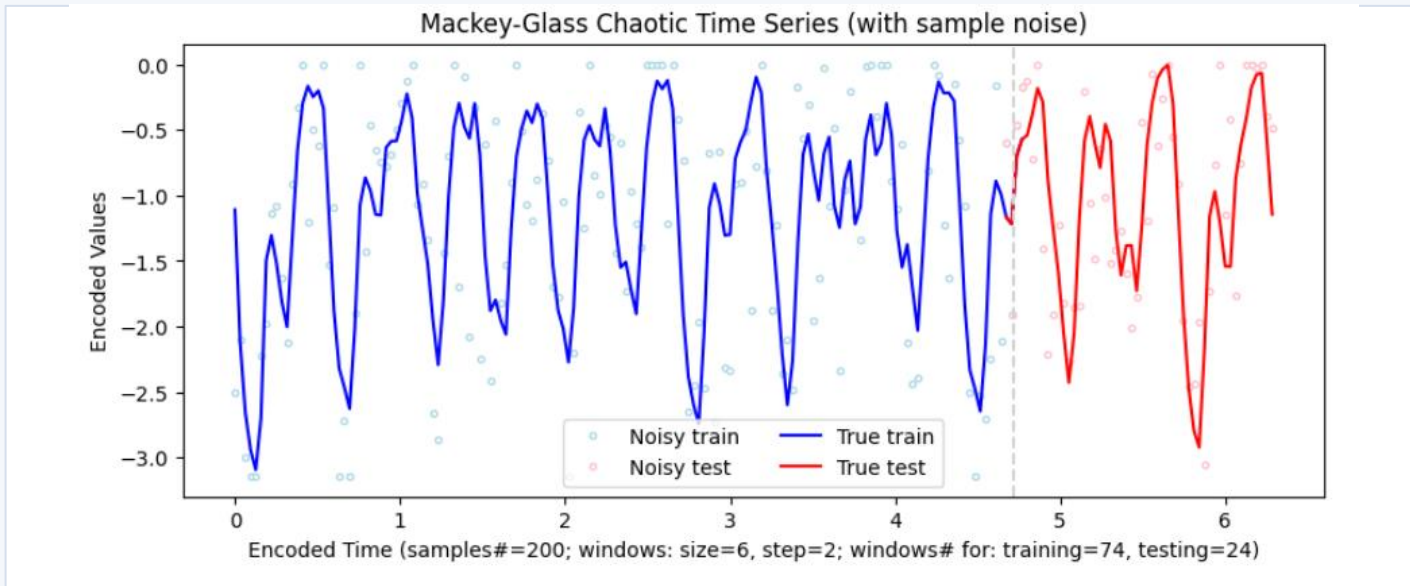


Fig. 6 — Mackey-Glass sequence ( $\tau = 30$ ) with added noise  $\sigma = 0.2$

## Data: time series

Not images, not tables — sequences. Order matters.

## Dynamics: chaotic

Mackey-Glass,  $\tau = 30$ . No easy decomposition into sinusoids.

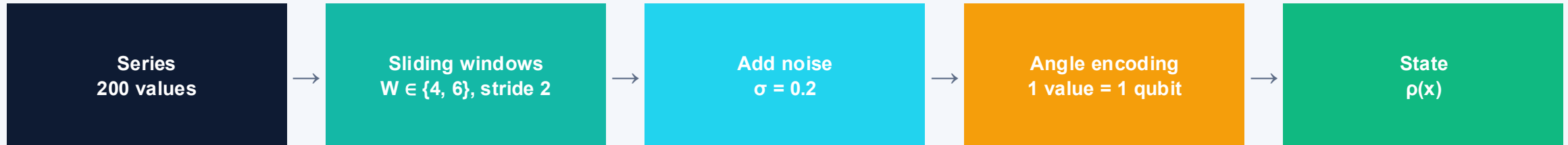
## Corruption: Gaussian noise

$x = s + \eta$ ,  $\eta \sim N(0, \sigma^2 I)$ ,  $\sigma = 0.2$ . Stochastic noise, deterministic signal.

**The task: learn the map  $x \mapsto \hat{s}$  — separate the deterministic signal from stochastic noise.**

# Series → time windows → quantum state

A quantum circuit does not accept a “series”. We must cut the signal into fixed-length vectors.



## Why windows?

The circuit has a fixed qubit count. A window of length  $W$  = one training example =  $W$  qubits.

## Why stride 2?

Overlapping windows increase the sample count and preserve continuity of local dynamics.

## Why angle encoding?

One value → one rotation angle on one qubit.  
Simple, shallow, NISQ-friendly.

The train/test split is **CHRONOLOGICAL** — the model never sees the future.

# The classical autoencoder — and why corrupt the input at all

## Plain autoencoder

$$\mathbf{s} \mapsto \mathbf{s}$$

The encoder squeezes the input through a bottleneck (latent); the decoder reconstructs it. It learns compression.

Problem: with enough capacity it can learn the identity function — simply copy input to output. It does not understand the data, it copies it.

## Denoising autoencoder

$$\mathbf{x} = \mathbf{s} + \boldsymbol{\eta} \mapsto \hat{\mathbf{s}}$$

We deliberately corrupt the INPUT but keep the CLEAN signal as the target. The model gets  $\mathbf{x}$  and must return  $\mathbf{s}$ .

Now copying is punished — it would copy the noise too. To minimize the error it must learn what is structure and what is accident.

## WHY ADD NOISE — in one sentence

Corruption breaks the input-output symmetry ( $\mathbf{s} \mapsto \mathbf{s}$  becomes  $\mathbf{x} \mapsto \mathbf{s}$ ). That is what forces the model to extract the signal manifold instead of memorizing samples. Noise is not an obstacle — it is the regularizer and the very point of the task.

# Quantum Denoising Autoencoder (QAE)

Same idea, but the bottleneck is realized physically — by partitioning the qubit register.

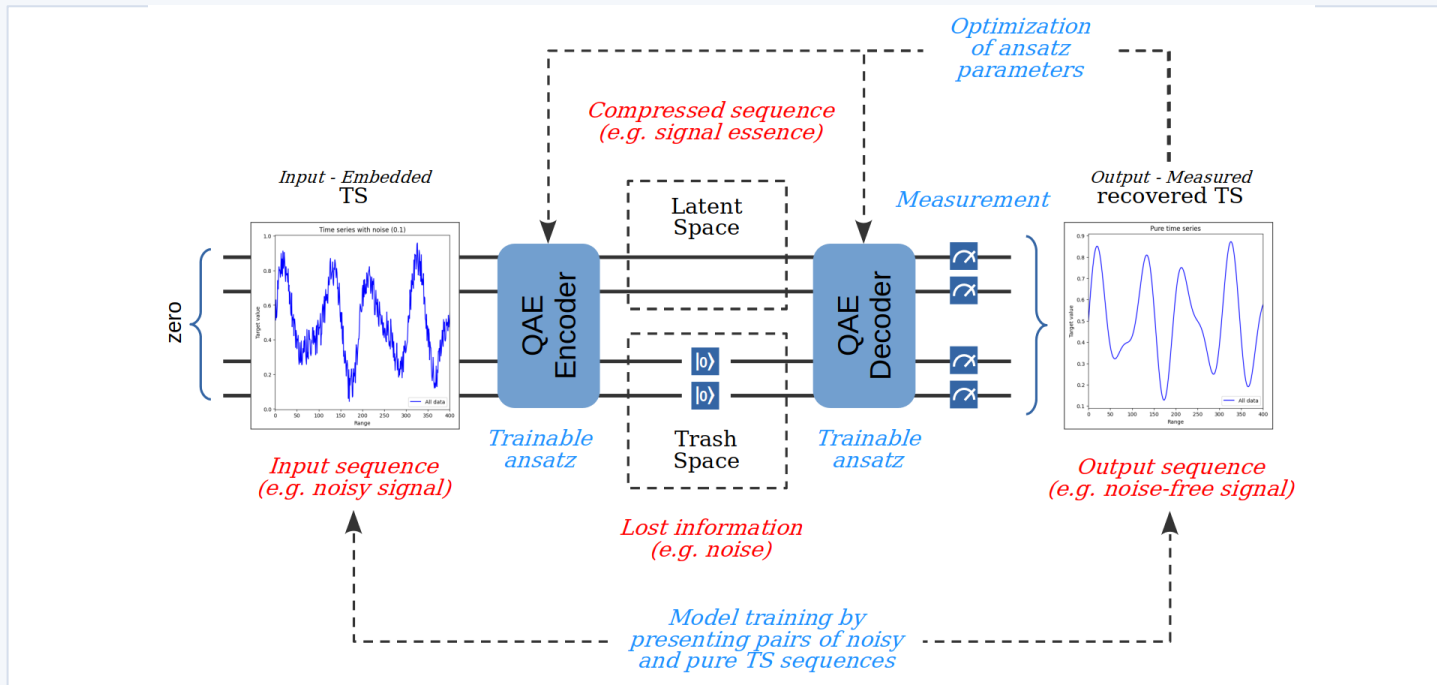


Fig. 1 — a typical denoising QAE for time series

## Encoding

The window enters the circuit as an N-qubit state:  $\rho(x)$ .

## Latent space

n qubits — the compressed representation of the signal.

## Trash space

$N - n$  qubits — where noise and redundancy are pushed.

## Trash reset

The key step: the trash register is reset to  $|0\rangle$ . Information is IRREVERSIBLY destroyed.

$$H \cong H_L \otimes H_T$$

# Symmetry — the axis along which our models differ

Before we meet the four architectures, let us fix one axis. They differ only in which symmetries they keep.

## Why symmetry MUST break

### Denoising is many-to-one

Many different  $x = s + \eta$  map to the same  $s$ . It is an irreversible, contractive map.

### A unitary is a bijection

$U^\dagger U = I$  — a unitary circuit loses nothing. By itself it CANNOT denoise.

### The one non-unitary element

Resetting the trash to  $|0\rangle$ . That is the model's ENTIRE budget of irreversibility.

## Three symmetries we may keep or break

### Structural

Is the encoder an inverted decoder? (encoder = decoder $^\dagger$ )

### Functional

Do we map  $s \mapsto s$  or  $x \mapsto s$ ? Adding noise breaks this by definition.

### Weight symmetry

Shared or independent encoder and decoder parameters?

### Training metric

Do we train on the OUTPUT, or on a proxy target in the latent space?

**HYPOTHESIS (the standard QML recipe): keep the symmetry — modularity, pretraining, encoder = decoder $^\dagger$  — because it cures barren plateaus. We test that recipe on a task that is inherently asymmetric.**

# Train everything at once, or pretrain the decoder first?

This is the axis of the whole paper. Same circuits, same capacity — the only difference is HOW we optimize.

## STRATEGY A · SINGLE-STAGE

### Train it all at once (Monolith)

The encoder (parameters  $\theta$ ) and decoder (parameters  $\phi$ ) have INDEPENDENT weights and are trained together, from scratch, from a single loss: MSE between the output and the clean signal  $s$ .

No pretraining. No imposed relation between encoder and decoder.

## STRATEGY B · TWO-STAGE

### Decoder first, then the rest (Mirror / Stacked / Sidekick)

STAGE 1: train the decoder separately on CLEAN data (Half-QAE) — it learns the manifold of clean dynamics.

STAGE 2: assemble encoder + decoder, feed NOISY data, fine-tune.

Variants: frozen (Stage 1 weights locked) or retrained (they keep learning).

### Why would anyone do B? — because it is the standard QML recipe

(1) Barren plateaus: pretraining gives a sensible initialization instead of a random one. (2) Unitary reversibility:  $U^\dagger U = I$ , so the inverted decoder IS a ready-made encoder — we get it for free. (3) Fewer parameters optimized at once. It is the quantum analogue of greedy layer-wise pretraining and transfer learning.

**We did not choose B because we believe it is better. We chose it because THE LITERATURE DOES — and that is exactly what we are testing.**

# Half-QAE — how we pretrain the decoder

The shared foundation of all three modular models: Mirror, Stacked and Sidekick.

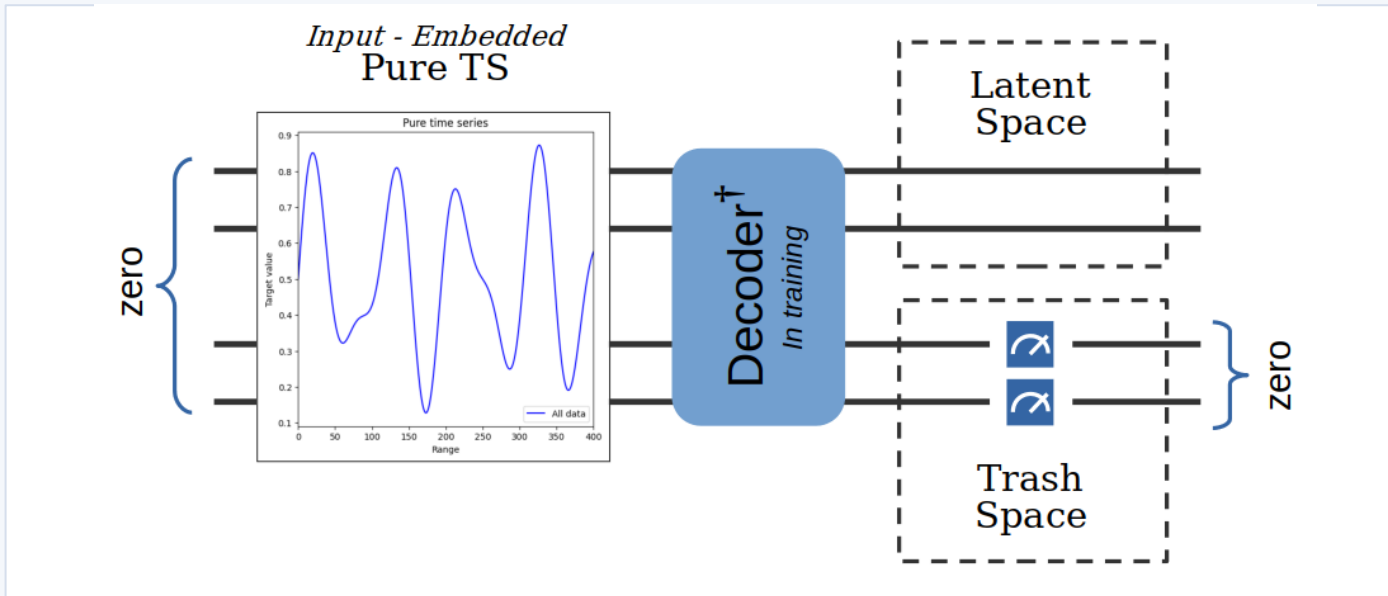


Fig. 2 — Half-QAE (inverted decoder)

## Input: CLEAN data

We feed  $p(s)$ , not  $p(x)$ . Stage 1 never sees noise.

## Goal: trash → vacuum

Optimize  $\varphi$  so the trash register is driven to  $|0\rangle$ . Measured by a SWAP test / fidelity.

## Payoff: a free encoder

By unitary reversibility: encoder = decoder $^\dagger$ . One trained circuit, two uses.

**AND HERE THE TROUBLE IS BORN: the trained unitary is BLIND to noise — it learned only clean dynamics.**



# Mirror — maximal symmetry

Encoder and decoder are the same unitary, merely inverted:  $\text{encoder} = \text{decoder}^\dagger$ . Weights are SHARED.

**WEIGHTS:** shared · **STRUCTURE:**  $\text{encoder} = \text{decoder}^\dagger$  · **LOSS:** SWAP on trash (Stage 1)

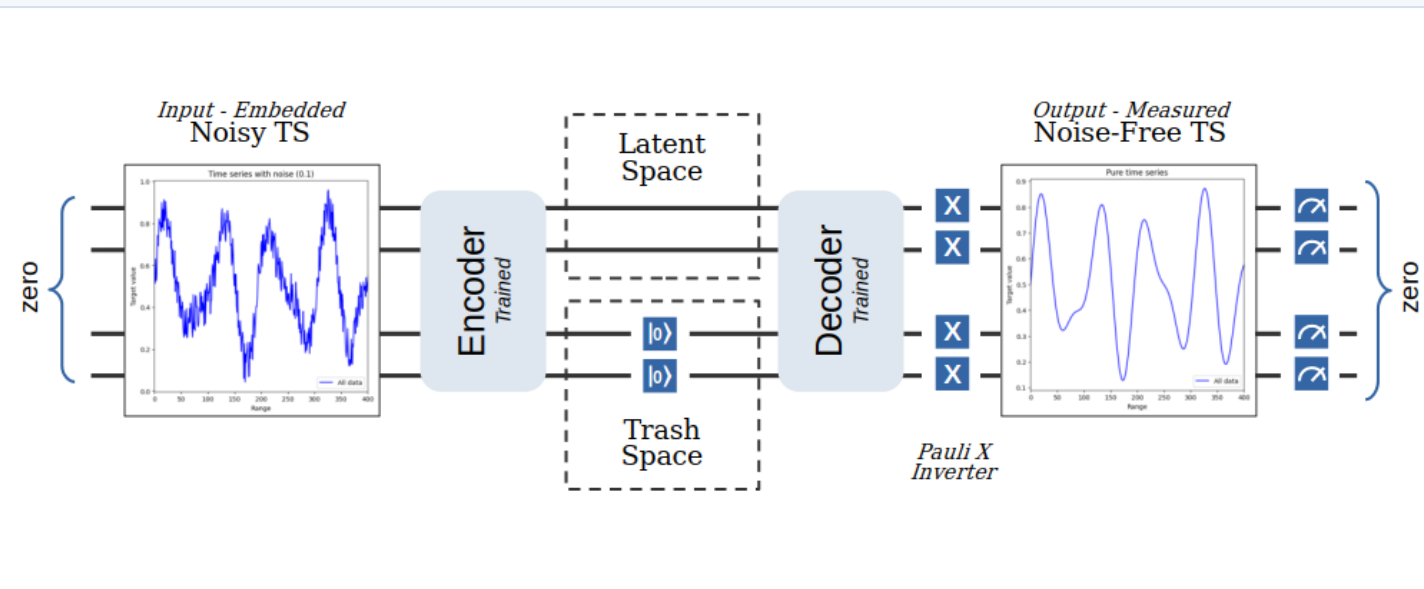


Fig. 3 — Mirror QAE (note the result must also be inverted)

## Stages

Effectively one — we train only half the circuit (Half-QAE).

## Strength

Fewest parameters, shortest runtime (295 s). A classic of lossy compression.

## Weakness

ZERO parameters dedicated to noise. All capacity goes to the clean manifold.

## THE SYMMETRY TRAP

$\Delta_{\text{test}} = 40.01\%$

Structural symmetry is a liability here, not elegance.

# Stacked — pretraining + tuning on the output

Stage 1: clean prior (Half-QAE). Stage 2: attach the encoder, feed noise, minimize MSE ON THE OUTPUT.

**WEIGHTS:** broken (retrained) · **STRUCTURE:** broken · **LOSS:** MSE on the OUTPUT

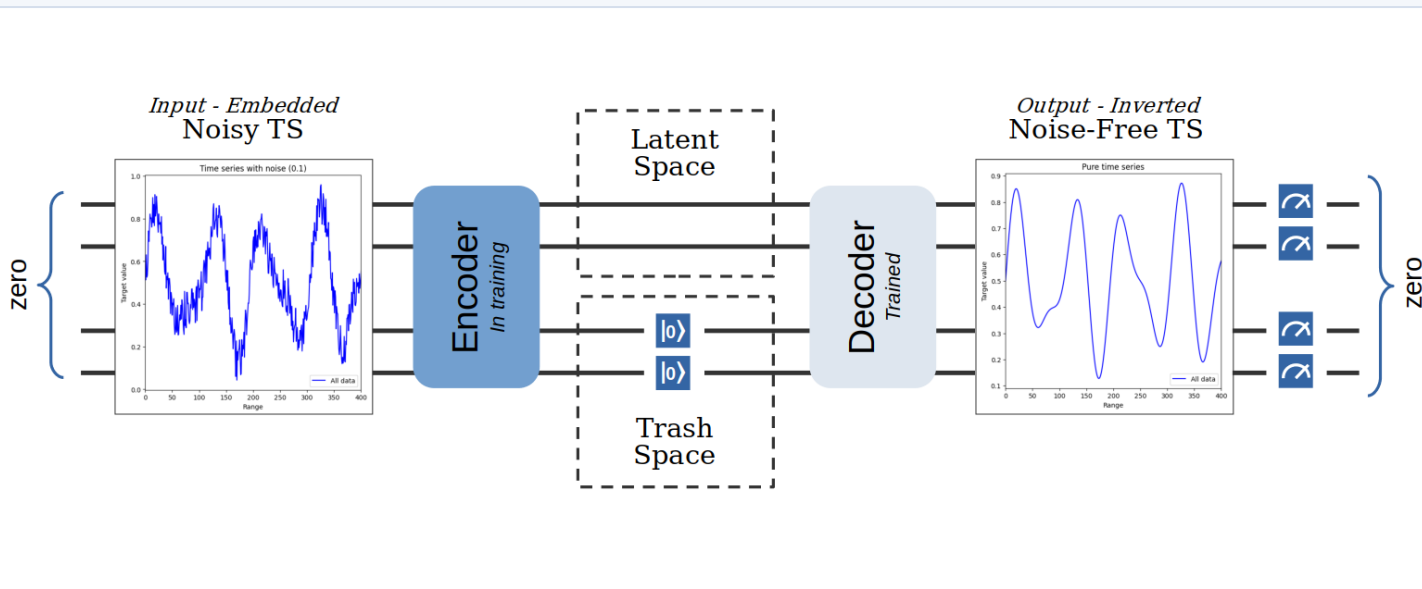


Fig. 4 — Stage 2 training of the Stacked QAE

## Stage-2 loss

MSE directly between reconstruction and clean target s. The metric = what we actually care about.

## Frozen variant

Stage-1 decoder weights locked.  $\Delta_{\text{test}} = 49.99\%$ .

## Retrained variant

The decoder keeps learning and can undo the Stage-1 bias.  $\Delta_{\text{test}} = 64.45\%$ .

## BEST MODULAR MODEL

$\Delta_{\text{test}} = 64.45\%$  (retrained)

It works **BECAUSE** it trains on the right metric.

# Sidekick — pretraining + latent synchronization

Almost identical to Stacked, with one difference — and that difference kills it.

**WEIGHTS:** broken · **STRUCTURE:** broken · **LOSS:** SWAP in the LATENT (proxy metric)

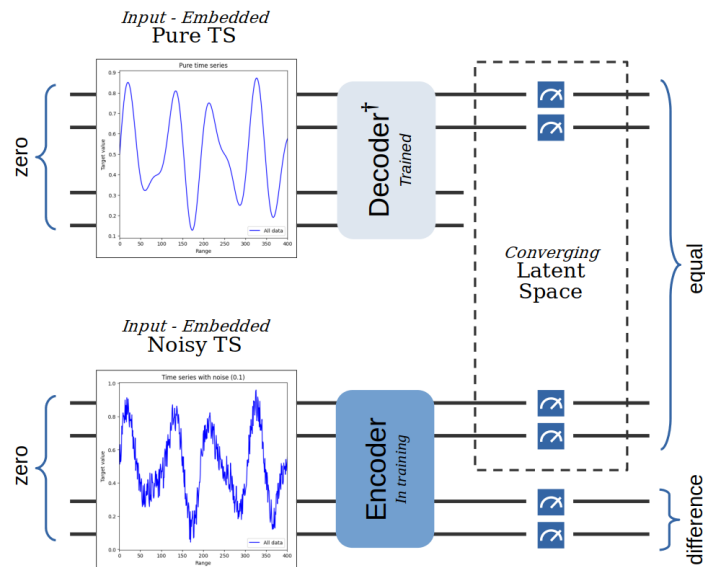


Fig. 5 — Stage 2 training of the “Sidekick” QAE

## Stage-2 loss

A SWAP test synchronizing the encoder latent with the clean-prior latent.

## What it never sees

It never compares the OUTPUT against the clean signal. It learns through a proxy metric.

## Consequence

Latent convergence  $\neq$  reconstruction quality. Huge spread:  $\pm 21.66$ .

## PROXY-METRIC DISCONNECT

$\Delta_{\text{test}} = 33.79\%$

The worst result despite the most sophisticated design.

# Monolith — co-evolutionary asymmetry

No pretraining. No imposed symmetry. Encoder and decoder learn together, but independently.

**WEIGHTS:** independent  $\theta, \phi$  · **STRUCTURE:** unconstrained · **LOSS:** MSE on the OUTPUT

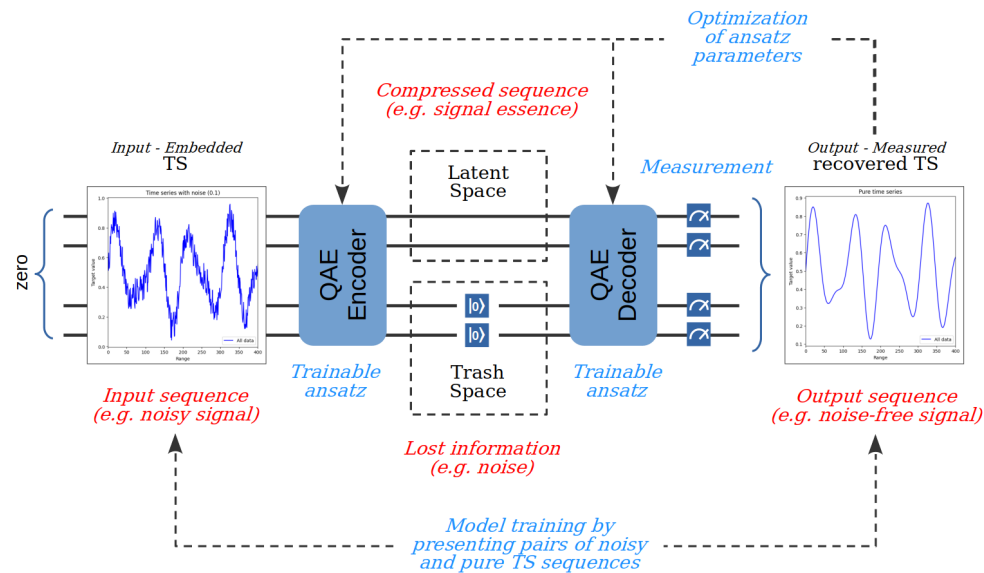


Fig. 1 — the full QAE trained in a single stage

## Parameters

$\theta$  and  $\phi$  fully independent — no encoder = decoder† constraint.

## Training

One stage, 300 epochs, loss = MSE on the output. 299 s — the fastest.

## Co-evolution

Encoder and decoder adapt to each other, seeing noise from the very start.

## THE WINNER

$\Delta_{\text{test}} = 67.15\% \pm 3.45$

Best, fastest and most stable — at the lowest cost.

# Four bets — the symmetry scorecard

Same data, same circuit capacity. The difference lies purely in which symmetries each model preserves.

| Architecture         | Stages | Weight symmetry  | Structural symmetry           | Stage-2 loss   |
|----------------------|--------|------------------|-------------------------------|----------------|
| Mirror               | 1      | preserved        | preserved ( $E = D^\dagger$ ) | —              |
| Stacked (frozen)     | 2      | partial (locked) | broken                        | MSE on output  |
| Stacked (retrained)  | 2      | broken           | broken                        | MSE on output  |
| Sidekick (frozen)    | 2      | partial (locked) | broken                        | SWAP in latent |
| Sidekick (retrained) | 2      | broken           | broken                        | SWAP in latent |
| Monolith             | 1      | broken           | unconstrained                 | MSE on output  |

The question for the rest of the talk: which column turns out to be decisive — the circuit structure, or the metric we train on?

# How we compared — fairly, under identical conditions

Every architecture got the same circuit capacity, the same data and the same evaluation protocol.

## Data

- Mackey-Glass,  $\tau = 30$
- 200 series values
- Windows  $W \in \{4, 6\}$ , stride 2
- Noise  $\sigma = 0.2$
- Chronological split

## Model

- 6 qubits
- 4 layers
- Latent/trash split 4–2
- Hardware-efficient ansatz
- Angle encoding

## Training & evaluation

- PennyLane, lightning.qubits
- Adam optimizer
- Median over 7 instances
- Metric:  $\Delta_{\text{test}}$  (%)
- Train  $\leftrightarrow$  test check (Pearson  $r$ )

Median over 7 instances — robust to the occasional convergence failures typical of variational circuits.

# Best configurations — performance vs. effort

Table 2 — all models: 6 qubits, 4 layers, 4–2 latent/trash split.  $\Delta_{\text{test}}$  = share of the injected noise removed: 100% = perfect recovery, 0% = no better than the noisy input, below 0 = the model made it worse. Brackets: spread of  $\Delta_{\text{test}}$  over 7 instances.

| Architecture         | Stage 1 | Stage 2 | Total epochs | $\Delta_{\text{test}}$ (%) | Time (s) |
|----------------------|---------|---------|--------------|----------------------------|----------|
| Monolith             | 0       | 300     | 300          | 67.15 ( $\pm 3.45$ )       | 299      |
| Stacked (retrained)  | 400     | 300     | 700          | 64.45 ( $\pm 4.31$ )       | 525      |
| Stacked (frozen)     | 600     | 500     | 1100         | 49.99 ( $\pm 6.50$ )       | 692      |
| Sidekick (frozen)    | 500     | 500     | 1000         | 47.81 ( $\pm 10.19$ )      | 657      |
| Mirror               | 500     | 100     | 600          | 40.01 ( $\pm 7.68$ )       | 295      |
| Sidekick (retrained) | 200     | 300     | 500          | 33.79 ( $\pm 21.66$ )      | 372      |

Monolith: the best score with ZERO pretraining epochs, the shortest runtime and the smallest spread.

Sidekick (retrained): the worst score and a  $\pm 21.66$  spread — despite the most complex design.

# And what does the MEAN across all regimes look like?

Table 1 — averaged over all structural variants ( $N = 35\text{--}42$  configurations per model), not just the single best one. Here  $r$  is NOT a quality score: it is the correlation between  $\Delta_{\text{train}}$  and  $\Delta_{\text{test}}$ , i.e. a check that our weight-selection heuristic generalizes.

| Model                | $\Delta_{\text{train}}$ (%) | $\Delta_{\text{test}}$ (%) | SD ( $\Delta_{\text{train}} - \Delta_{\text{test}}$ ) | $r$ (Pearson) |
|----------------------|-----------------------------|----------------------------|-------------------------------------------------------|---------------|
| Stacked (retrained)  | +55.54                      | +61.73                     | 3.46                                                  | 0.45          |
| Monolith             | +45.71                      | +53.70                     | 4.24                                                  | 0.99          |
| Stacked (frozen)     | +43.19                      | +46.98                     | 3.60                                                  | 0.88          |
| Sidekick (frozen)    | +33.61                      | +44.46                     | 7.30                                                  | 0.69          |
| Mirror               | +28.97                      | +32.78                     | 7.09                                                  | 0.79          |
| Sidekick (retrained) | +11.23                      | +21.86                     | 24.63                                                 | 0.78          |

## Note 1 — who really wins?

On the mean, Stacked (retrained) leads with 61.73%. Monolith wins only in the BEST configuration (67.15%) — and does it with zero pretraining.

## Note 2 — read $r$ and SD together

Stacked (retrained) has the LOWEST generalization gap of all (SD = 3.46). Its  $r = 0.45$  is range restriction: results cluster tightly at the top, so there is little variance left to correlate. Saturation, not instability.

## Note 3 — high $r$ can be worthless

Sidekick (retrained) has  $r = 0.78$  only because its spread is enormous (SD = 24.63). It is reliably predictable — and reliably bad. The hierarchy holds in BOTH tables.

**NISQ takeaway: Monolith wins on efficiency (0 pretraining epochs), Stacked (retrained) on the mean and on the generalization gap. Both break weight symmetry. Both beat Mirror and Sidekick.**



# What we were able to show

Three classic QML heuristics — modularity, symmetry, pretraining — fail on an inherently asymmetric task.

1

### Pretraining can hurt

A prior learned on CLEAN data is blind to noise. Stage 2 wastes budget undoing its own bias — negative transfer.

2

### Symmetry is a liability

The encoder = decoder† constraint (Mirror) strips away parameters that could model noise. Denoising REQUIRES symmetry breaking.

3

### Proxy metrics deceive

Sidekick converges in the latent and loses at the output. SWAP-test convergence  $\neq$  reconstruction quality.

4

### Co-evolutionary asymmetry wins

Independent, jointly trained weights (Monolith) give the best result at the lowest computational cost.

Denoising quality is governed by **OPTIMIZATION SYMMETRY**, not by circuit depth.

# Thank you

---

Future work:

- Scaling with signal entropy and longer windows
- Validation on quantum hardware (beyond the simulator)
- NISQ compromise: Stacked (retrained) as a warm start where Monolith risks a barren plateau

Acknowledgments: Paweł Gora and the Quantum AI Foundation · AIntern 2025 program

Co-authors: J. L. Cybulski, J. Zwoniarski, A. Strąg, S. Zając